Algorítmica I · Ingeniería Informática · Curso 2026/2027

EPD 4 · Proyecto evaluable Bloque I

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \cdot \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix}$$

Los grimorios de Frieren

Multiplicación de matrices mágicas:
algoritmo iterativo frente a algoritmo recursivo

Autor: Sergio

Fecha: 24 de octubre de 2026

Proyecto Java: `EPD04` (NetBeans)

«El hechizo de detección tiene una precisión del 99%. Los grandes descubrimientos de la historia los hicieron magos que confiaron en ese 1% restante.» — Frieren

Índice

1. Introducción al problema

- 1.1 El producto de matrices
- 1.2 Objetivos de la misión y dónde se responden
- 1.3 Estructura del proyecto Java
- 1.4 Entorno experimental

2. Generador de matrices mágicas

3. Explicación del algoritmo iterativo (objetivo 1)

- 3.1 Código
- 3.2 Explicación detallada
- 3.3 Traza de un ejemplo

4. Explicación del algoritmo recursivo (objetivo 2)

- 4.1 Idea: divide y vencerás sobre las tres dimensiones
- 4.2 Código
- 4.3 Explicación detallada
- 4.4 Árbol de recursión de un ejemplo
- 4.5 Corrección y terminación

5. Evidencias experimentales (objetivo 3)

- 5.1 Sistema de experimentación automática
- 5.2 Metodología de medida
- 5.3 Evidencias de corrección

6. Tabla de tiempos y gráfica comparativa (objetivos 3 y 4)

7. Análisis de complejidad (objetivo 5)

- 7.1 Algoritmo iterativo: conteo de operaciones elementales
- 7.2 Algoritmo recursivo: ecuación de recurrencia
- 7.3 Complejidad espacial
- 7.4 Relación entre la teoría y los resultados

8. Comparación razonada entre algoritmos (objetivo 5)

9. Conclusiones finales (objetivo 5)

Bibliografía

Anexo A. Ejecución del proyecto

1. Introducción al problema

1.1 El producto de matrices

Frieren representa cada hechizo de sus grimorios como una matriz de energía mágica. Combinar dos hechizos consiste en **multiplicar sus matrices**. Dadas una matriz A de dimensión $m \times n$ y una matriz B de dimensión $n \times p$, el producto $C = A \cdot B$ es una matriz $m \times p$ cuyos elementos son:

$$C[i][j] = \Sigma_{k=0}^{n-1} A[i][k] \cdot B[k][j] \qquad 0 \leq i < m,\ 0 \leq j < p$$

El producto solo está definido si el número de columnas de A coincide con el número de filas de B (dimensiones **compatibles**). Cada elemento de C es el producto escalar de una fila de A por una columna de B, que necesita n multiplicaciones. Por tanto, cualquier algoritmo que aplique directamente la definición realiza ***m·n·p* productos escalares**, es decir, n^3 cuando las matrices son cuadradas. La pregunta de la misión es **cómo organizar ese trabajo**: con bucles (iterativo) o descomponiendo el problema en subproblemas más pequeños (recursivo), y qué consecuencias tiene cada opción en el tiempo de ejecución.

1.2 Objetivos de la misión y dónde se responden

#	Objetivo	Apartados del informe
1	Multiplicación con un algoritmo iterativo, explicando el código	§3
2	Multiplicación con un algoritmo recursivo, explicando el código	§4
3	Coste temporal de cada algoritmo para diversas dimensiones	§5 y §6.1 (tabla de tiempos)
4	Gráfica única con la evolución del coste de ambos algoritmos	§6.2 (gráfica comparativa)
5	Complejidad de cada algoritmo, comparación y elección razonada	§7, §8 y §9

1.3 Estructura del proyecto Java

El proyecto NetBeans `EPD04` contiene un único paquete, `epd04`, con clases de utilidad formadas **exclusivamente por métodos estáticos**, como exige el enunciado:

Clase	Responsabilidad
`EPD04`	Programa principal. Al pulsar *Run* ejecuta automáticamente las cuatro fases (validación, experimentación, resultados y gráfica). Los resultados se muestran por consola y la gráfica en una ventana.
`GeneradorMatrices`	Genera matrices aleatorias de enteros a partir del número de filas y columnas recibido como parámetro (tarea 1).
`MultiplicacionMatrices`	Los dos hechizos: `multiplicarIterativo` y `multiplicarRecursivo`, más la validación de dimensiones (tarea 2).
`Experimentos`	Batería automática de experimentos, medición de tiempos, calentamiento de la JVM, variable de consumo, tabla de tiempos y análisis empírico por consola (tareas 3 y 4).
`Grafica`	Dibuja en memoria la gráfica comparativa con Java2D (sin bibliotecas externas) y la muestra en una ventana al final de la ejecución. No escribe ningún fichero.
`ResultadoExperimento`	*Record* inmutable con los tiempos de una instancia (mínimo y media de cada algoritmo).

1.4 Entorno experimental

Todas las medidas de este informe se han obtenido en un único equipo y en una sola ejecución del programa: procesador **AMD Ryzen 5 5600X** (6 núcleos; caché L1 de datos de 32 KB y L2 de 512 KB por núcleo, L3 compartida de 32 MB), **Windows 11 Pro** y **Java HotSpot 25** (proyecto compilado con nivel de lenguaje Java 20). La resolución del temporizador `System.nanoTime()` en este sistema es de 100 ns. Al ser medidas *a posteriori*, los valores absolutos dependen de la máquina, pero las tendencias y las proporciones entre algoritmos son las que se analizan.

2. Generador de matrices mágicas

El método `GeneradorMatrices.generarMatrizAleatoria(filas, columnas)` crea una matriz *filas* × *columnas* y rellena cada celda con un entero aleatorio de energía en el intervalo [1, 9]. Una sobrecarga permite indicar también el intervalo `[minimo, maximo]`, lo que lo hace reutilizable en otros contextos.

```
public static int[][] generarMatrizAleatoria(int filas, int columnas, int minimo, int maxi
    if (filas <= 0 || columnas <= 0) {          // una matriz necesita al menos 1 fila y 1
        throw new IllegalArgumentException("Las dimensiones deben ser positivas...");
    }
    if (minimo > maximo) { throw new IllegalArgumentException(...); }
    int[][] matriz = new int[filas][columnas];
    int amplitud = maximo - minimo + 1;             // cantidad de valores posibles
    for (int i = 0; i < filas; i++) {
        for (int j = 0; j < columnas; j++) {
            matriz[i][j] = minimo + azar.nextInt(amplitud);  // valor en [minimo, maximo]
        }
    }
    return matriz;
}
```

- **Parametrización:** filas y columnas son parámetros de entrada, así que genera matrices cuadradas o rectangulares de cualquier tamaño. Las dimensiones no positivas se rechazan con una excepción descriptiva.
- **Aleatoriedad reproducible:** se usa un único objeto `java.util.Random` compartido. El programa le asigna al inicio una semilla basada en la hora y la muestra por consola, de modo que cualquier ejecución puede repetirse con `establecerSemilla(semilla)`.
- **Rango de valores:** con valores en [1, 9] el mayor elemento posible de un producto 800 × 800 es 800·81 = 64 800, muy lejos del máximo de un `int` ($\approx 2{,}1 \cdot 10^9$), así que no hay desbordamiento.
- **Coste:** Θ(filas·columnas), ya que cada celda se visita una sola vez.

Las matrices de cada experimento se **generan aleatoriamente en memoria**, en el propio código, justo antes de medir. No se usan ficheros: cada ejecución del programa trabaja con matrices nuevas, y la semilla mostrada al inicio permite repetir exactamente cualquier ejecución. La generación no forma parte del tiempo medido.

3. Explicación del algoritmo iterativo (objetivo 1)

3.1 Código

```
1  public static int[][] multiplicarIterativo(int[][] a, int[][] b) {
2      validarCompatibles(a, b);
3
4      int m = a.length;      // filas de A = filas de C
5      int n = b.length;      // columnas de A = filas de B (dimensión común)
6      int p = b[0].length;   // columnas de B = columnas de C
7
8      int[][] c = new int[m][p];
9
10     for (int i = 0; i < m; i++) {
11         int[] filaA = a[i];   // referencia a la fila i de A
12         for (int j = 0; j < p; j++) {
13             // Producto escalar de la fila i de A por la columna j de B
14             int suma = 0;
15             for (int k = 0; k < n; k++) {
16                 suma += filaA[k] * b[k][j];
17             }
18             c[i][j] = suma;
19         }
20     }
21     return c;
22 }
```

3.2 Explicación detallada

- **Línea 2, validación.** `validarCompatibles` comprueba que ambas matrices no son nulas ni vacías, que todas sus filas tienen la misma longitud y que `a[0].length == b.length`. Si no se cumple, lanza una `IllegalArgumentException` que indica las dimensiones recibidas. Así el algoritmo **gestiona las dimensiones incompatibles** en lugar de fallar con un índice fuera de rango.
- **Líneas 4–6, dimensiones.** Se extraen m, n y p. El algoritmo admite matrices rectangulares, no solo cuadradas.
- **Línea 8, resultado.** Se reserva C ($m \times p$). Java inicializa sus celdas a 0.
- **Bucle `i` (líneas 10–20):** recorre las filas de A, que son también las filas de C. En la línea 11 se guarda la referencia `filaA = a[i]` para no repetir el acceso `a[i]` en cada iteración del bucle interno. Es una pequeña optimización que no cambia el algoritmo.
- **Bucle `j` (líneas 12–19):** recorre las columnas de B, que son también las columnas de C. El par (i, j) identifica la celda de C que se va a calcular.
- **Bucle `k` (líneas 15–17):** calcula el producto escalar Σ_k A[i][k]·B[k][j] sobre la dimensión común.

  La suma se acumula en una variable local, `suma`, que el compilador mantiene en un registro, y se escribe en C una sola vez (línea 18) en lugar de n veces.
- **Línea 21:** se devuelve la matriz producto.

> El cuerpo del bucle más interno (línea 16) se ejecuta exactamente ***m·n·p* veces** (n^3 para matrices cuadradas), sean cuales sean los valores de las matrices. Por tanto **no hay mejor ni peor caso**: el algoritmo es determinista en su número de operaciones.

3.3 Traza de un ejemplo

Para A = [[1,2,3],[4,5,6]] (2×3) y B = [[7,8],[9,10],[11,12]] (3×2), el bucle `k` calcula cada celda así:

(i, j)	Iteraciones de k (suma parcial)	C[i][j]
(0, 0)	1·7 = 7 → 7 + 2·9 = 25 → 25 + 3·11 = 58	58
(0, 1)	1·8 = 8 → 8 + 2·10 = 28 → 28 + 3·12 = 64	64
(1, 0)	4·7 = 28 → 28 + 5·9 = 73 → 73 + 6·11 = 139	139
(1, 1)	4·8 = 32 → 32 + 5·10 = 82 → 82 + 6·12 = 154	154

Se realizan 2·3·2 = 12 multiplicaciones escalares, tal como predice la fórmula $m \cdot n \cdot p$. Este mismo ejemplo es el primero que comprueba el programa (fase 1.1, apartado 5.3).

4. Explicación del algoritmo recursivo (objetivo 2)

4.1 Idea: divide y vencerás sobre las tres dimensiones

El producto de matrices es un problema con tres dimensiones (m, n, p). El algoritmo recursivo aplica la técnica de **divide y vencerás** basada en la multiplicación por bloques: en cada llamada **parte por la mitad la dimensión más grande** y resuelve recursivamente los dos subproblemas resultantes. Las tres formas de dividir se basan en identidades del álgebra de matrices por bloques:

Dimensión mayor	Descomposición	Subproblemas
1) m (filas de A)	$[C_1 ; C_2] = [A_1 ; A_2] \cdot B$	$C_1 = A_1 \cdot B$ y $C_2 = A_2 \cdot B$ (bloques de C disjuntos)
2) p (columnas de B)	$[C_1 \mid C_2] = A \cdot [B_1 \mid B_2]$	$C_1 = A \cdot B_1$ y $C_2 = A \cdot B_2$ (bloques de C disjuntos)
3) n (dimensión común)	$C = [A_1 \mid A_2] \cdot [B_1 ; B_2] = A_1 \cdot B_1 + A_2 \cdot B_2$	Ambos subproblemas se **acumulan** sobre el mismo bloque de C

Caso base: cuando $m = n = p = 1$ el subproblema es un único producto escalar, `C[i][j] += A[i][k]·B[k][j]`. La suma de los n términos de cada celda no la hace ningún bucle: la hacen las distintas hojas del árbol de recursión, que acumulan sobre la misma celda.

Decisiones de diseño:

- **Sin copias de submatrices.** Cada subproblema se describe con tres desplazamientos (`filaA`, `comun`, `colB`) y tres tamaños (`m`, `n`, `p`) sobre las matrices originales. Dividir cuesta $O(1)$ y combinar no cuesta nada, porque las hojas escriben directamente en su posición de C.
- **Dimensiones arbitrarias.** Al partir en $\lfloor x/2 \rfloor$ y $\lceil x/2 \rceil$ no hace falta que las dimensiones sean potencias de 2 ni que las matrices sean cuadradas, así que no se rellena con ceros (*padding*).
- **Totalmente recursivo.** No hay ningún bucle en el algoritmo: el caso base es un único producto escalar, sin sustituciones iterativas.
- **Partir la dimensión mayor** mantiene los subproblemas lo más «cúbicos» posible. La profundidad del árbol queda en $\lceil \log_2 m \rceil + \lceil \log_2 n \rceil + \lceil \log_2 p \rceil$, unas 30 llamadas para matrices 800×800, sin riesgo de desbordar la pila.

4.2 Código

```
1  public static int[][] multiplicarRecursivo(int[][] a, int[][] b) {
2      validarCompatibles(a, b);
3      int m = a.length;
4      int n = b.length;
5      int p = b[0].length;
6      int[][] c = new int[m][p];                          // inicializada a ceros
7      multiplicarRecursivo(a, b, c, 0, 0, 0, m, n, p);    // llamada inicial: problema compl
8      return c;
9  }
10
11 private static void multiplicarRecursivo(int[][] a, int[][] b, int[][] c,
12         int filaA, int comun, int colB, int m, int n, int p) {
13     if (m == 1 && n == 1 && p == 1) {                                  // CASO BASE
14         c[filaA][colB] += a[filaA][comun] * b[comun][colB];
15         return;
16     }
17     if (m >= n && m >= p) {                                            // 1) partir filas de
18         int mitad = m / 2;
19         multiplicarRecursivo(a, b, c, filaA, comun, colB, mitad, n, p);
20         multiplicarRecursivo(a, b, c, filaA + mitad, comun, colB, m - mitad, n, p);
21     } else if (p >= n) {                                               // 2) partir columnas
22         int mitad = p / 2;
23         multiplicarRecursivo(a, b, c, filaA, comun, colB, m, n, mitad);
24         multiplicarRecursivo(a, b, c, filaA, comun, colB + mitad, m, n, p - mitad);
25     } else {                                                           // 3) partir la dimer
26         int mitad = n / 2;
27         multiplicarRecursivo(a, b, c, filaA, comun, colB, m, mitad, p);
28         multiplicarRecursivo(a, b, c, filaA, comun + mitad, colB, m, n - mitad, p);
29     }
30 }
```

4.3 Explicación detallada

- **Método público (líneas 1–9).** Valida las dimensiones con el mismo método que el iterativo, reserva C a ceros (necesario porque las hojas acumulan con `+=`) y lanza la recursión sobre el problema completo: bloque de A desde (0,0) de tamaño $m \times n$ y bloque de B desde (0,0) de tamaño $n \times p$.
- **Parámetros del método privado (líneas 11–12).** El subproblema actual es «acumular en C el producto del bloque de A que empieza en (`filaA`, `comun`) y mide $m \times n$ por el bloque de B que empieza en (`comun`, `colB`) y mide $n \times p$». El bloque de C afectado empieza en (`filaA`, `colB`).
- **Caso base (líneas 13–16).** Bloques 1×1: un producto escalar que se acumula en su celda de C.
- **Caso 1 (líneas 17–20).** Si m es la mayor dimensión, se resuelven por separado la mitad superior de filas (`mitad` filas desde `filaA`) y la mitad inferior (`m - mitad` filas desde `filaA + mitad`). Cada mitad escribe en filas distintas de C.
- **Caso 2 (líneas 21–24).** Si p es la mayor, se reparten las columnas de B (y de C) en dos mitades, desplazando `colB`.
- **Caso 3 (líneas 25–28).** Si la mayor es n, se divide la dimensión común desplazando `comun`. Las dos llamadas actualizan el **mismo** bloque de C y la suma $A_1 \cdot B_1 + A_2 \cdot B_2$ se obtiene por acumulación.
- Las condiciones están ordenadas para que el caso elegido sea siempre la dimensión estrictamente necesaria: si no se cumple el caso 1, la mayor es p o n; si además $p \geq n$, la mayor es p; si no, es n.

4.4 Árbol de recursión de un ejemplo

Para dos matrices 2×2 la llamada inicial es (m,n,p) = (2,2,2). Primero se parte *m*, luego *p* y por último *n*. El árbol tiene 8 = 2·2·2 hojas (una por producto escalar) y 15 = 2·8 − 1 nodos en total:

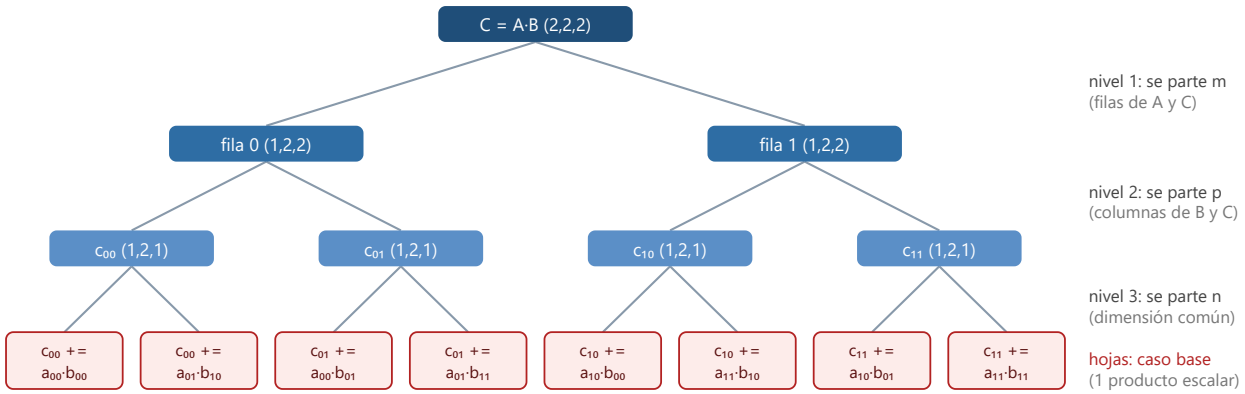


Figura 1. Árbol de recursión de `multiplicarRecursivo` para A(2×2)·B(2×2). Entre paréntesis, (m, n, p) del subproblema.

4.5 Corrección y terminación

- **Terminación.** Si no estamos en el caso base, la dimensión mayor es ≥ 2, así que sus mitades ⌊x/2⌋ y ⌈x/2⌉ son ≥ 1 y estrictamente menores que *x*. La suma *m*+*n*+*p* decrece en cada llamada y está acotada inferiormente por 3, de modo que siempre se alcanza el caso base.
- **Corrección** (inducción sobre *m*·*n*·*p*). El caso base calcula correctamente su único término. En el paso inductivo, cada división reparte el conjunto de ternas (*i*, *k*, *j*) del subproblema en dos subconjuntos disjuntos que lo cubren por completo, y por las identidades de la tabla de §4.1 la suma de sus contribuciones es exactamente el producto del bloque. Por tanto **cada término A[i][k]·B[k][j] se suma una y solo una vez** en C[i][j].
- **Comprobación empírica.** El programa compara ambos algoritmos en matrices de dimensiones variadas (incluidas no cuadradas y no potencias de 2) y en todas las instancias del experimento. Los resultados son idénticos en todos los casos (§5.3).

5. Evidencias experimentales (objetivo 3)

5.1 Sistema de experimentación automática

El profesor solo tiene que pulsar **Run** en NetBeans. El método `main` de `EPD04` ejecuta sin intervención manual estas fases. Los resultados aparecen en la consola y la gráfica en una ventana:

Fase	Qué hace
1. Validación	Ejemplo resuelto a mano (2×3·3×2), ejemplo con matrices aleatorias, 10 productos rectangulares aleatorios comparando ambos algoritmos y contando las llamadas recursivas, y detección de dimensiones incompatibles.
2. Experimentos	Calienta la JVM y después, para cada dimensión $n \in \{10, 25, 50, 75, 100, 150, 200, 250, 300, 400, 500, 600, 700, 800\}$, **genera aleatoriamente** dos matrices $n \times n$ (A y B), las multiplica varias veces con ambos algoritmos midiendo el tiempo y comprueba que los resultados coinciden. El progreso se muestra por consola.
3. Resultados	Muestra por consola la tabla de tiempos ordenada por dimensión y el análisis empírico de la complejidad: T/n^3, exponente local, pendiente log-log y ajuste $c \cdot n^3$.
4. Gráfica	Abre una ventana con la gráfica comparativa de ambos algoritmos para todas las instancias medidas (la misma que la figura 2 de este informe).

Las dimensiones a analizar están en la constante `Experimentos.DIMENSIONES`. Para añadir instancias basta con modificar esa lista. La ejecución completa tarda unos 36 segundos en el equipo de pruebas.

5.2 Metodología de medida

Se sigue la metodología de la EPD 2 (medida *a posteriori* con temporizador de nanosegundos), con algunas precauciones adicionales propias de la JVM:

- **Solo se mide el algoritmo.** Se toma `System.nanoTime()` inmediatamente antes y después de la llamada. La generación de las matrices queda fuera.
- **Mismas condiciones.** Todas las repeticiones de una dimensión usan exactamente las mismas matrices A y B, generadas una sola vez.
- **Varias repeticiones y tiempo mínimo.** Cada algoritmo se ejecuta 50 veces para $n \le 50$, 20 para $n \le 150$, 10 para $n \le 300$, 5 para $n \le 500$ y 3 para el resto. La medida principal es el **mínimo**, la menos afectada por interrupciones del sistema operativo o del recolector de basura. También se registra la media.
- **Calentamiento (*warm-up*).** Antes de medir, ambos métodos se ejecutan 300 veces con matrices 64×64 para que el compilador JIT los traduzca a código máquina optimizado. Sin este paso, las primeras medidas incluirían el tiempo de interpretación.
- **Ejecuciones alternadas.** En cada repetición se ejecuta primero el iterativo y después el recursivo, para que cualquier perturbación temporal afecte por igual a ambos.
- **Variable de consumo.** Cada producto calculado (en el calentamiento y en las mediciones) se «consume» sumando uno de sus valores a la variable estática `consumo`, que se imprime al final del programa. El compilador JIT elimina el código cuyo resultado nunca se usa (*eliminación de código muerto*). Si el resultado de la multiplicación se descartara, el JIT podría ahorrarse parte del cálculo y se mediría menos trabajo del real. Con la variable de consumo el resultado siempre se usa, así que el JIT está obligado a hacer el cálculo completo y las medidas son fiables.

- **Llamada a `System.gc()`** antes de cada serie de medidas, para reducir la probabilidad de que el recolector interrumpa una medición.

5.3 Evidencias de corrección (salida real del programa, fase 1)

```
1.1 Ejemplo resuelto a mano (2x3 · 3x2):
    Iterativo (2x2):            Recursivo (2x2):
      |   58   64 |              |   58   64 |
      |  139  154 |              |  139  154 |
    Coincide con el resultado esperado: iterativo sí, recursivo sí

1.3 Matrices aleatorias de dimensiones variadas (A: m x n, B: n x p):
    Dimensiones          Iter = Rec    Llamadas rec.   2·m·n·p − 1
    1x1 · 1x1            sí            1               1
    1x7 · 7x1            sí            13              13
    5x1 · 1x3            sí            29              29
    2x3 · 3x4            sí            47              47
    7x5 · 5x3            sí            209             209
    13x17 · 17x11        sí            4861            4861
    31x64 · 64x9         sí            35711           35711
    64x33 · 33x20        sí            84479           84479
    100x50 · 50x200      sí            1999999         1999999
    128x128 · 128x128    sí            4194303         4194303

1.4 Dimensiones incompatibles (2x3 · 2x3):
    Iterativo -> excepción controlada: Matrices incompatibles para el producto: 2x3 · 2x3 (...)
    Recursivo -> excepción controlada: Matrices incompatibles para el producto: 2x3 · 2x3 (...)
```

La columna «Llamadas rec.» la obtiene el método `contarLlamadasRecursivas`, que reproduce la misma estrategia de división. Coincide en todos los casos con la fórmula teórica $2 \cdot m \cdot n \cdot p - 1$ que se demuestra en §7.2. Además, en las 14 instancias del experimento ambos algoritmos devolvieron matrices idénticas (columna «Correcto» de la tabla 1).

6. Tabla de tiempos y gráfica comparativa (objetivos 3 y 4)

6.1 Tabla de tiempos obtenidos

n (n×n)	Rep.	Iterativo (ms)		Recursivo (ms)		Rec / Iter	Correcto
		mínimo	media	mínimo	media		
10	50	0,0008	0,0013	0,0059	0,0072	7,38	sí
25	50	0,0089	0,0106	0,0888	0,0990	9,98	sí
50	50	0,0658	0,0717	0,7075	0,7503	10,75	sí
75	20	0,2189	0,2494	2,4153	2,7822	11,03	sí
100	20	0,5304	0,6452	5,9699	7,2688	11,26	sí
150	20	2,0004	2,3283	20,3204	23,3575	10,16	sí
200	10	4,6513	5,1611	47,9620	52,4352	10,31	sí
250	10	10,5678	11,6304	91,5699	97,0606	8,66	sí
300	10	25,0550	26,8852	164,0293	178,3782	6,55	sí
400	5	67,7801	70,3886	396,8221	405,3720	5,85	sí
500	5	136,2258	140,8899	742,8907	799,8415	5,45	sí
600	3	237,6048	246,0865	1 357,3427	1 379,1453	5,71	sí
700	3	382,7671	388,0930	2 169,6870	2 211,8959	5,67	sí
800	3	585,4647	595,5552	3 156,1982	3 160,7012	5,39	sí

Tabla 1. Tiempos de ejecución (milisegundos) del producto de dos matrices n×n. «Rec / Iter» compara los tiempos mínimos e indica cuántas veces es más lento el recursivo. Valores copiados de la fase 3 de la salida por consola del programa.

La media supera al mínimo como mucho en un 22% para n ≤ 250 (salvo en n = 10) y en menos de un 9% para n ≥ 300, lo que indica medidas estables. Los valores de $n = 10$ están cerca de la resolución del temporizador (100 ns) y deben tomarse como orientativos.

6.2 Gráfica comparativa

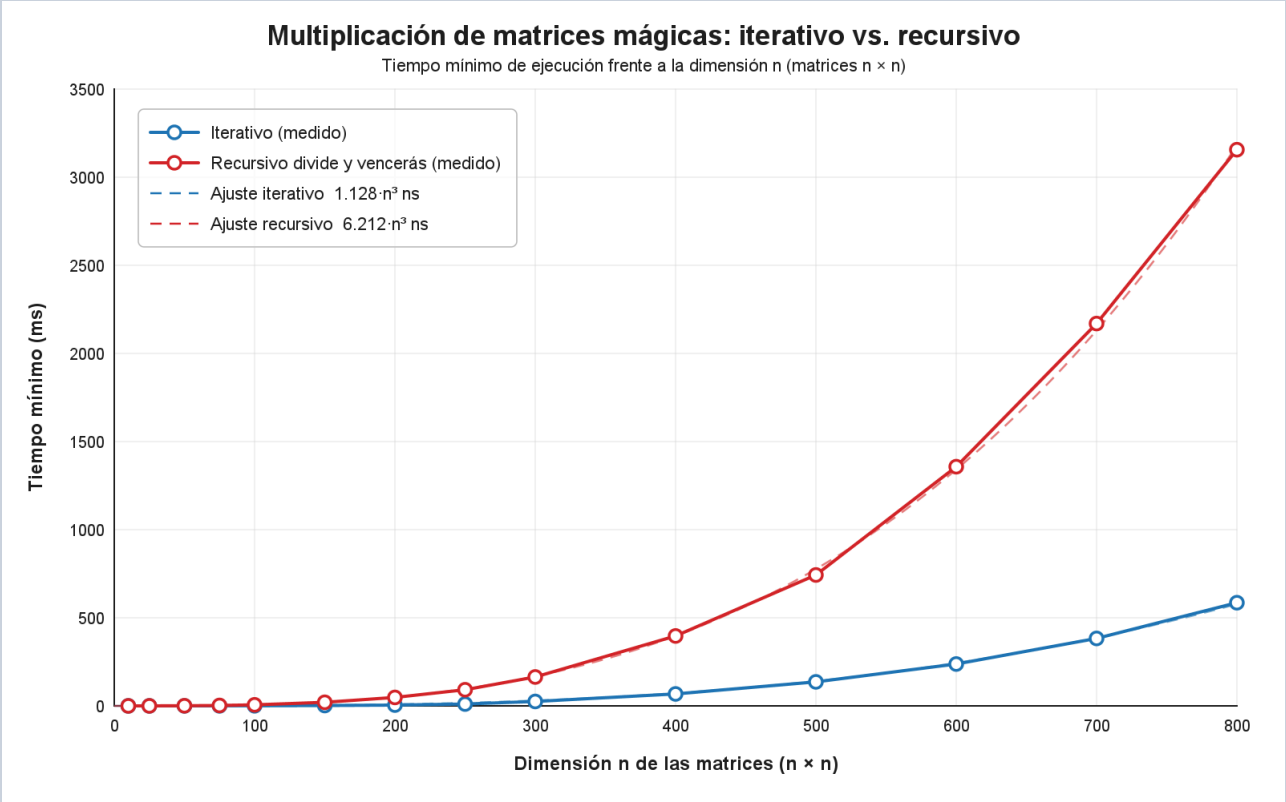


Figura 2. Tiempo mínimo de ambos algoritmos para las 14 instancias analizadas, dibujada por la clase `Grafica` del proyecto con los tiempos de la tabla 1. Es la gráfica que muestra el programa en su ventana final (fase 4). En trazo discontinuo, el ajuste por mínimos cuadrados $T(n) \approx c \cdot n^3$ de cada algoritmo.

La gráfica muestra que **ambas curvas crecen de forma cúbica**: las curvas teóricas $c \cdot n^3$ (discontinuas) se superponen casi exactamente a las medidas. También muestra que el recursivo es **siempre más lento** y que la distancia entre ambos crece en valor absoluto con *n*, aunque la proporción entre ellos se mantiene acotada (tabla 1).

7. Análisis de complejidad (objetivo 5)

Se usa el modelo de **operaciones elementales (OE)** de la EPD 2: cada operación aritmética, asignación, comparación, acceso a un vector o matriz, llamada o retorno cuenta como 1 OE. El tamaño del problema son las dimensiones (*m*, *n*, *p*), con *m* = *n* = *p* en el caso cuadrado.

7.1 Algoritmo iterativo: conteo de operaciones elementales

Se cuenta de dentro hacia fuera, sobre el código de §3.1:

- **Cuerpo interno** (línea 16) `suma += filaA[k] * b[k][j]`: 3 accesos (`filaA[k]`, `b[k]`, `[j]`) + 1 producto + 1 suma + 1 asignación = **6 OE**.
- **Bucle k**: 1 (inicialización) + *n*·(1 comparación + 6 + 2 del incremento) + 1 (comparación final) = **9n + 2**.
- **Una iteración del bucle j**: 1 (comparación) + 1 (`suma = 0`) + (9n + 2) + 3 (`c[i][j] = suma`) + 2 (incremento) = 9n + 9. **Bucle j completo**: 1 + p·(9n + 9) + 1 = **9np + 9p + 2**.
- **Una iteración del bucle i**: 1 + 2 (`filaA = a[i]`) + (9np + 9p + 2) + 2 = 9np + 9p + 7. **Bucle i completo**: 1 + m·(9np + 9p + 7) + 1.

$$T_{iter}(m, n, p) = 9 \cdot m \cdot n \cdot p + 9 \cdot m \cdot p + 7 \cdot m + 2 + \Theta(m + n) + \Theta(m \cdot p) + c$$

Los términos $\Theta(m + n)$ y $\Theta(m \cdot p)$ corresponden a la validación (recorre las filas de A y B) y a la reserva de *C*, que la JVM inicializa a ceros. Para matrices cuadradas:

$$T_{iter}(n) = 9n^3 + O(n^2) \implies \mathbf{T_{iter}(n) \in \Theta(n^3)} \quad \text{(en general, } \Theta(m \cdot n \cdot p))$$

El número de operaciones no depende de los valores de las matrices, sino solo de sus dimensiones, así que **mejor caso = peor caso = caso medio = $\Theta(n^3)$**.

7.2 Algoritmo recursivo: ecuación de recurrencia

Sea *T*(m,n,p) el coste del método privado. Un caso base hace un número constante de OE, c_h (3 comparaciones, 6 accesos, producto, suma, asignación y retorno: unas 15 OE). Un nodo interno hace un trabajo constante c_i (comparaciones para elegir el caso, el cálculo de `mitad`, las sumas de desplazamientos y las dos llamadas) más el de sus dos hijos:

$$T(1,1,1) = c_h \qquad T(m,n,p) = T(\text{subproblema}_1) + T(\text{subproblema}_2) + c_i$$

a) Caso cuadrado con n = 2^k: teorema maestro

Partiendo de (n,n,n), el algoritmo parte *m*, después *p* y después *n*. Tras tres niveles se obtienen **8 subproblemas** de tamaño (n/2, n/2, n/2), con 7 nodos internos de coste constante (como en la figura 1):

$$T(n) = 8 \cdot T(n/2) + 7 \cdot c_i \quad \implies \quad a = 8,\ b = 2,\ f(n) = \Theta(1)$$

Como $\log_b a = \log_2 8 = 3$ y $f(n) = \Theta(1) = O(n^{3-\varepsilon})$ para cualquier $0 < \varepsilon \leq 3$, se aplica el **caso 1 del teorema maestro**: $\mathbf{T(n) \in \Theta(n^{\log_2 8}) = \Theta(n^3)}$. Desarrollando la recurrencia se obtiene además la solución exacta:

$$T(n) = 8^k \cdot c_h + 7c_i \cdot (1 + 8 + \ldots + 8^{k-1}) = c_h \cdot n^3 + c_i \cdot (n^3 - 1)$$

b) Caso general (cualquier m, n, p): conteo del árbol de recursión

- Cada división reparte las ternas (i, k, j) del subproblema entre dos hijos disjuntos, y cada hoja tiene exactamente una terna. Por tanto el árbol tiene **L = m·n·p hojas**.
- Cada nodo interno tiene exactamente 2 hijos (árbol binario estricto), y un árbol así con L hojas tiene L − 1 nodos internos. En total hay **2·m·n·p − 1 llamadas**, cifra que el programa verifica (§5.3).

$$T_{rec}(m,n,p) = c_h \cdot m \cdot n \cdot p + c_i \cdot (m \cdot n \cdot p - 1) + \Theta(m \cdot p) + \Theta(m + n) \implies \mathbf{T_{rec} \in \Theta(m \cdot n \cdot p)},$$
$$\mathbf{\Theta(n^3)} \text{ si son cuadradas}$$

Como en el iterativo, el trabajo no depende de los valores: **mejor caso = peor caso = Θ(n³)**. Ambos algoritmos pertenecen a la **misma clase de complejidad**. Difieren en la constante: el iterativo hace unas 9 OE por producto escalar y el recursivo unas $c_h + c_i \approx 30$, y además cada una de sus $2n^3 - 1$ llamadas implica crear un marco de pila con 9 parámetros, lo que en la práctica es más caro que una OE simple.

7.3 Complejidad espacial

- **Iterativo:** Θ(m·p) para el resultado y Θ(1) de memoria adicional (los índices y la suma).
- **Recursivo:** Θ(m·p) para el resultado más la pila de recursión, cuya profundidad es $\lceil \log_2 m \rceil + \lceil \log_2 n \rceil + \lceil \log_2 p \rceil \in \mathbf{\Theta(\log n)}$. Para n = 800 son 30 marcos, muy lejos del límite de la pila. No se copian submatrices, así que no hay memoria adicional $\Theta(n^2)$.

7.4 Relación entre la teoría y los resultados

Si $T(n) \approx c \cdot n^3$, el cociente $T(n)/n^3$ debe ser aproximadamente constante y, al doblar *n*, el tiempo debe multiplicarse por $2^3 = 8$. El exponente local $\log(T_2/T_1)/\log(n_2/n_1)$ y la pendiente de la recta log T frente a log n deben valer aproximadamente 3. El programa calcula estos indicadores y los muestra por consola (fase 3):

n	Iter. T/n^3 (ns)	Rec. T/n^3 (ns)	Exp. iter.	Exp. rec.
25	0,570	5,683	2,63	2,96
50	0,526	5,660	2,89	2,99
75	0,519	5,725	2,96	3,03
100	0,530	5,970	3,08	3,15
150	0,593	6,021	3,27	3,02
200	0,581	5,995	2,93	2,99
250	0,676	5,860	3,68	2,90
300	0,928	6,075	4,73	3,20
400	1,059	6,200	3,46	3,07
500	1,090	5,943	3,13	2,81
600	1,100	6,284	3,05	3,31
700	1,116	6,326	3,09	3,04
800	1,143	6,164	3,18	2,81

Exponente local: entre n y la dimensión anterior.

Tabla 2. Tiempo normalizado por n^3 y exponente empírico local (a partir de los tiempos mínimos).

Indicador	Iterativo	Recursivo	Teoría
Factor al doblar n: 100 → 200	× 8,77	× 8,03	× 8
Factor al doblar n: 200 → 400	× 14,57	× 8,27	× 8
Factor al doblar n: 400 → 800	× 8,64	× 7,95	× 8
Pendiente log-log, n de 50 a 250	3,15	3,03	3
Pendiente log-log, n ≥ 400	3,10	3,03	3
Pendiente log-log, n ≥ 100 (todo el rango)	3,43	3,02	3
Ajuste por mínimos cuadrados $T(n) \approx c \cdot n^3$	$1{,}128 \cdot n^3$ ns	$6{,}212 \cdot n^3$ ns	$c \cdot n^3$

Tabla 3. Indicadores empíricos del orden de crecimiento.

Interpretación

- **Recursivo: cúbico de forma casi perfecta.** T/n^3 se mantiene entre 5,7 y 6,3 ns en todo el rango, al doblar *n* el tiempo se multiplica por 7,9–8,3 y la pendiente log-log es 3,02–3,03. Los datos confirman la predicción $T_{rec}(n) = (c_h + c_i) \cdot n^3 - c_i$, con $c_h + c_i \approx 6$ ns por producto escalar en esta máquina.
- **Iterativo: cúbico en cada tramo, con un salto de constante.** Para $n \le 250$, $T/n^3 \approx 0{,}52$–$0{,}68$ ns y la pendiente es 3,15. Para $n \ge 400$, $T/n^3 \approx 1{,}06$–$1{,}14$ ns y la pendiente es 3,10. Entre ambos tramos (n = 300) **la constante se duplica**, y por eso la pendiente global sale 3,43 y el factor 200→400 es ×14,6. **No es un cambio de complejidad**, sino un efecto de la **jerarquía de memoria**: el bucle interno recorre *B* por **columnas** (`b[k][j]` con *k* variable), saltando de una fila a otra en cada paso. Mientras *B* cabe en la caché L2 ($4n^2$ bytes: 250 KB para n = 250, frente a 512 KB de L2), esos accesos son baratos. A partir de n ≈ 300 (352 KB, que con A y C ya no caben bien) y sobre todo n ≥ 400 (625 KB), cada recorrido de *B* debe traerse de la L3, y el coste por operación pasa de ≈ 0,5 a ≈ 1 ns. Una vez superada la L2, el comportamiento vuelve a ser prácticamente cúbico.

- **El recursivo no sufre ese salto** porque divide y vencerás es un algoritmo *cache-oblivious*: al dividir, los subproblemas acaban cabiendo en la caché L1 sea cual sea *n*, y su coste por operación es estable. Por eso la proporción Rec/Iter baja de ≈ 10–11 ($n \leq 200$) a ≈ 5,4–5,9 ($n \geq 400$).
- **Ajuste global.** Las curvas $c \cdot n^3$ de la figura 2 se ajustan a las medidas en todo el rango, lo que confirma visualmente que **ambos algoritmos son $\Theta(n^3)$**. Lo que los distingue es la **constante oculta**: 1,13 frente a 6,21 ns por n^3, es decir, el recursivo es unas 5,5 veces más lento para matrices grandes.

8. Comparación razonada entre algoritmos (objetivo 5)

Criterio	Iterativo (3 bucles)	Recursivo (divide y vencerás)
Complejidad temporal	$\Theta(m \cdot n \cdot p)$; $\Theta(n^3)$	$\Theta(m \cdot n \cdot p)$; $\Theta(n^3)$
Mejor / peor caso	Iguales (no depende de los datos)	Iguales (no depende de los datos)
Productos escalares	$m \cdot n \cdot p$	$m \cdot n \cdot p$ (los mismos)
Sobrecoste de control	Una iteración de bucle por producto (≈ 9 OE)	$2 \cdot m \cdot n \cdot p - 1$ llamadas (≈ 2 por producto), cada una con 9 parámetros
Constante medida	≈ 0,52–0,68 $ns \cdot n^3$ ($n \leq 250$); ≈ 1,06–1,14 $ns \cdot n^3$ ($n \geq 400$)	≈ 5,7–6,3 $ns \cdot n^3$ en todo el rango
Tiempo relativo	**Referencia (más rápido)**	5,4–11,3 veces más lento
Memoria adicional	$\Theta(1)$	$\Theta(\log n)$ de pila
Uso de la caché	Recorre B por columnas: empeora al superar la L2	Buena localidad (*cache-oblivious*): rendimiento estable
Sencillez	Muy simple; traslación directa de la definición	Más compleja: desplazamientos, tres casos de división, acumulación
Posibles mejoras	Orden de bucles i-k-j o división en bloques (*tiling*) para mejorar la caché	Base de Strassen (7 productos en vez de 8: $\Theta(n^{2,81})$) y de la paralelización (los subproblemas de los casos 1 y 2 son independientes)

Tabla 4. Comparación entre ambos algoritmos.

¿Qué método debe elegir Frieren?

Los dos hechizos calculan el mismo resultado y realizan exactamente los mismos $m \cdot n \cdot p$ productos escalares, de modo que **tienen la misma complejidad asintótica, $\Theta(n^3)$**. Ninguno escala mejor que el otro: al doblar el tamaño del grimorio, ambos tardan unas 8 veces más. La diferencia está en la **constante multiplicativa**. El recursivo dedica la mayor parte de su tiempo a gestionar las $2n^3 - 1$ llamadas (apilar parámetros, comparar dimensiones, retornar) y no a multiplicar, mientras que el iterativo solo paga un incremento y una comparación de bucle por producto.

> **Recomendación:** para combinar grimorios en la práctica, Frieren debe usar el **algoritmo iterativo**. Es entre 5,4 y 11 veces más rápido en todas las dimensiones medidas (por ejemplo, 0,59 s frente a 3,16 s para matrices 800×800), usa memoria adicional constante y es más sencillo de verificar y mantener.

El algoritmo recursivo no es inútil. Su estructura de divide y vencerás es la que permite **reducir la complejidad** con el método de Strassen, que sustituye los 8 subproductos por 7 y obtiene $\Theta(n^{\log_2 7}) \approx \Theta(n^{2,81})$. También permite repartir los subproblemas independientes entre varios núcleos y aprovecha mejor la caché, como muestra la estabilidad de su T/n^3. En implementaciones profesionales se combinan ambas ideas: se divide recursivamente hasta bloques de unas decenas de elementos y esos bloques se multiplican con bucles. Esa versión híbrida no se ha usado aquí porque el enunciado exige una solución plenamente recursiva, sin sustituciones iterativas.

9. Conclusiones finales

1. Se han implementado en Java, con métodos estáticos, dos algoritmos correctos de multiplicación de matrices válidos para cualquier dimensión compatible: uno **iterativo** de tres bucles anidados y uno **recursivo de divide y vencerás** que parte la mayor de las tres dimensiones sin copiar submatrices. Su corrección se ha comprobado con un ejemplo resuelto a mano, con 10 productos rectangulares aleatorios y con las 14 instancias del experimento.
2. El proyecto es **totalmente automático**: genera las matrices aleatoriamente en el código, mide ambos algoritmos con una metodología controlada (calentamiento de la JVM, variable de consumo, repeticiones, mínimo y alternancia) muestra por consola la tabla de tiempos y el análisis empírico, y termina mostrando la gráfica comparativa en una ventana, todo sin intervención manual.
3. **Ambos algoritmos son $\Theta(n^3)$** ($\Theta(m \cdot n \cdot p)$ en general), en todos los casos. Para el iterativo se ha demostrado contando operaciones elementales ($9n^3 + O(n^2)$). Para el recursivo, con el teorema maestro ($T(n) = 8T(n/2) + \Theta(1)$) y con el conteo exacto de su árbol de recursión ($2 \cdot m \cdot n \cdot p - 1$ llamadas, verificado por el programa).
4. Los experimentos confirman la teoría: los tiempos se multiplican por ≈ 8 al doblar *n*, las pendientes log-log valen ≈ 3 y el ajuste $c \cdot n^3$ reproduce las medidas. El salto de constante del iterativo en torno a $n = 300$ se explica por la jerarquía de memoria (la matriz B deja de caber en la caché L2), no por un cambio de complejidad.
5. Con la misma complejidad, el factor decisivo es la constante: el recursivo es **5,4–11 veces más lento** por el coste de sus llamadas. Para Frieren, **el algoritmo iterativo es el método más adecuado**. La versión recursiva tiene valor como base de algoritmos asintóticamente mejores (Strassen) y de versiones paralelas o por bloques.

> **Lección del grimorio:** la notación asintótica dice cómo crece el coste, pero no cuánto cuesta cada operación. Dos hechizos de la misma complejidad pueden diferir en un orden de magnitud en la práctica. Por eso el análisis teórico (*a priori*) y la medida experimental (*a posteriori*) son complementarios.

Bibliografía

- Cormen, T. H., Leiserson, C. E., Rivest, R. L. y Stein, C. *Introduction to Algorithms*. MIT Press. Capítulo 4: divide y vencerás, multiplicación de matrices y teorema maestro.
- Brassard, G. y Bratley, P. *Fundamentos de Algoritmia*. Prentice Hall.
- Bhargava, A. Y. *Grokking Algorithms*, 2.ª ed. Manning Publications, 2024.
- Algorítmica I, Curso 2026/2027. Enunciados de las EPD 1–4 (análisis de algoritmos, medición de tiempos y comparación de algoritmos).
- Documentación oficial de Oracle Java: `https://docs.oracle.com/en/java/`

Anexo A. Ejecución del proyecto

1. Abrir el proyecto `EPD04` en NetBeans (*File → Open Project*).

2. Pulsar **Run**. La clase principal es `epd04.EPD04`. No hace falta ningún argumento ni fichero previo.
3. La consola muestra las fases de validación, progreso de los experimentos y tabla de tiempos con el análisis empírico. Al final se abre una ventana con la gráfica comparativa; al cerrarla, termina el programa.
4. El programa no lee ni escribe ningún fichero: las matrices se generan aleatoriamente en memoria y la gráfica se dibuja también en memoria.